

高级语言程序设计

实验报告

南开大学 软件工程

姓名： 张敏

学号： 2413182

2026 年 5 月 10 日

目录

一、 作业题目	1
二、 开发软件与底层环境	1
三、 课题要求与核心指标	1
四、 主要流程与核心算法架构设计	2
1. 主循环 (Game Loop) 管道与事件捕获	2
2. 算法突破 1: 降维分离式 AABB 碰撞阻挡系统	2
3. 算法突破 2: 高阶实体交互与动量传递 (龟壳机制)	3
4. 算法突破 3: 复杂动画与经典抛物线模拟	3
五、 单元测试与边界验证	4
1. 核心交互测试矩阵	4
2. 缺陷修复与系统性能复盘	4
六、 重大收获与技术反思	5
1. 数据驱动架构: JSON 解析的降维打击	5
2. 像素级实战: 极其考验基本功的“抠坐标”纹理映射	5
3. 内存泄漏的梦魇与虚析构函数的救赎	6
七、 AI 工具使用披露声明	6

高级语言程序设计实验报告

一、 作业题目

本项目选定题目为：基于 C++ 与 Qt 框架的高级 2D 平台跳跃游戏引擎——Super Mario。

本项目旨在彻底脱离传统的控制台黑框字符界面，利用纯粹的面向对象编程思想（OOP），从底层逻辑开始构建一个完整的游戏运行环境。不仅要求实现基础的像素图形渲染，更核心的技术挑战在于：自研一套高精度的 AABB 物理碰撞系统、多层级的有限状态机（FSM）、跨平台的异步多媒体引擎，以及基于本地 JSON 解析的数据驱动关卡系统。通过本项目，期望深度锤炼 C++ 在处理复杂实时交互系统时的内存管理、多态架构及高性能计算能力。

二、 开发软件与底层环境

- 开发工具：Qt Creator 11.0
- 核心语言：C++（深度应用 Lambda 表达式、基于范围的 for 循环、智能指针与右值引用思想）。
- 关键图形库与框架：
 - Qt Graphics View Framework: 摒弃低效的 QPainter 直接绘制, 采用基于场景 (QGraphicsScene) 和图元 (QGraphicsItem) 的高性能渲染树结构。
 - Qt Multimedia: 构建了双规音频系统, 利用 QSoundEffect 实现近乎零延迟的动作音效 (如跳跃、金币、踩踏), 利用 QMediaPlayer 控制长段 BGM。
 - QJsonDocument: 负责庞大关卡地图的持久化数据序列化与反序列化。

三、 课题要求与核心指标

根据大作业大纲要求，本项目在架构设计之初即设立了以下严苛的技术准则：

1. 深度面向对象与多态抽象：严禁过程化的面向过程硬编码。所有实体类（如玩家 Player、怪物 Enemy、互动地块 TileItem、特效道具 Mushroom）必须抽象并继承自统一图形基类，利用虚函数重载实现多态更新。
2. 自研独立物理引擎：禁止调用 Box2D 等现成第三方物理库。必须自行通过牛顿力学离散化公式，实现重力场加速、惯性摩擦减速、抛物线运动及 AABB (Axis-Aligned Bounding Box) 碰撞检测。

3. 彻底的数据与逻辑解耦：环境坐标、怪物分布等硬数据必须全部剥离出 C++ 源码，以 JSON 格式动态挂载，实现“修改地图无需重新编译 C++ 代码”的工业级开发标准。

四、 主要流程与核心算法架构设计

1. 主循环 (Game Loop) 管道与事件捕获

在大型游戏开发中，主循环是整个系统的核心。本项目由 `GameWindow` 类作为中央调度器，抛弃了低效的事件阻塞机制，采用 `QTimer` 驱动精确的心跳时钟中断（设置为 16 毫秒，即锁定约 60FPS 满帧运行）。在主循环 `gameLoop()` 内部，每一帧严格执行以下流水线操作：

1. 无阻塞输入防抖：通过 `keyPressEvent` 实时截获按键并标记为布尔状态（如 `keyLeft=true`），彻底消除了操作系统键盘驱动自带的长按连击延迟。
2. 状态机演算：角色根据物理状态（站立、行走、跳跃、下落）结合外界输入，完成次态的转移。
3. X/Y 轴物理结算：先结算横向速度与阻挡，再结算垂直重力与支撑，避免复杂的二维斜角计算。
4. 对象生命周期管理：遍历对象池，清理被踩扁的敌人或被顶碎的砖块，释放堆区内存。
5. 摄像机平滑追踪：计算玩家最新相对坐标，调用视图控制器执行 `centerOn`。

2. 算法突破 1：降维分离式 AABB 碰撞阻挡系统

在 2D 平台游戏中，将 X 轴和 Y 轴合并进行碰撞检测会导致极难处理的“斜角穿模”和“卡墙”现象。本项目采用降维打击策略：先处理水平，后处理垂直，并在检测包围盒（Bounding Box）时引入了核心的“内向容差收缩算法”。

```
1 // 第一步：在 X 轴步进玩家坐标
2 player->setPos(player->x() + player->x_vel, player->y());
3
4 // 第二步：提取包围盒。利用 adjusted(0, 2, 0, -2) 极度关键！
5 // 强制收缩包围盒上下边缘2个像素，防止在平坦地面上移动时，因浮点数精度或
  图元拼接缝隙产生误撞墙判定
6 QRectF xCheckRect = player->sceneBoundingRect().adjusted(0, 2, 0, -2);
7
8 // 第三步：遍历静态环境物体并执行阻挡与坐标回退
9 for (auto it : solidItems) {
10     if (xCheckRect.intersects(it->sceneBoundingRect())) {
11         QRectF r = it->sceneBoundingRect();
12         // 发生穿透，强制将玩家推回到物体边缘外侧
```

```
13         if (player->x_vel > 0) player->setPos(r.left() - pWidth, player  
            ->y());  
14         else if (player->x_vel < 0) player->setPos(r.right(), player->y  
            ());  
15  
16         player->x_vel = 0; // 动能截断，物理引擎认定该轴方向速度归零  
17     }  
18 }
```

Listing 1: 带有像素级容差修正的 X 轴碰撞防卡墙逻辑

3. 算法突破 2：高阶实体交互与动量传递（龟壳机制）

游戏中的逻辑并非简单的玩家与静态物品的交互。项目中实现了复杂的 NPC 间状态传递机制。例如，当玩家踩踏乌龟后，乌龟会进入 SHELL_IDLE（静止龟壳）状态。玩家再次触碰会赋予其极大的横向速度，并能利用其击杀其他怪物。

```
1 // 踢出的龟壳击杀其他敌人的高速碰撞判定  
2 if (e->state == Enemy::SHELL_SLIDING) {  
3     for (int j = 0; j < enemies.size(); ++j) {  
4         if (i == j) continue; // 不与自己进行碰撞判定  
5         Enemy* e2 = enemies[j];  
6         if (e2->state != Enemy::SQUISHED && e->collidesWithItem(e2)) {  
7             // 发生次级碰撞，被击中敌人死亡，触发加分并播放踩踏音效  
8             e2->dieToShell();  
9             score += 500;  
10            sfxStomp->play();  
11        }  
12    }  
13 }
```

Listing 2: 龟壳的动量传递与环境清场逻辑

4. 算法突破 3：复杂动画与经典抛物线模拟

对于诸如马里奥死亡坠渊的经典画面，项目中采用了拟真物理建模。玩家死亡瞬间并非直接消失，而是冻结键盘输入，首先施加一个瞬间的逆向加速度（反重力），使其向上弹起，随后在帧刷新中持续施加向下重力，形成完美的抛物线轨迹，最后触发场景重载。

```
1 if (player->isDead) {  
2     stateTimer++;  
3     // 经典的死亡抛物线：前 30 帧施加反重力向上弹起，随后转为正常重力自由落体  
4     player->y_vel += (stateTimer < 30) ? C::ANTI_GRAVITY : C::GRAVITY;
```

```
5     player->setPos(player->x(), player->y() + player->y_vel);
6     // 跌出屏幕底端后，扣除生命，触发重置
7     if (stateTimer > 180) {
8         lives--;
9         currentState = (lives > 0) ? LOADING : GAMEOVER;
10    }
11 }
```

Listing 3: 玩家死亡抛物线物理模拟

五、单元测试与边界验证

1. 核心交互测试矩阵

针对游戏引擎中极易引发崩溃、死锁或越界行为的关键模块，设计了严密的单元边界测试：

表 1: 表 1：物理系统与多态交互逻辑测试案例

输入/触发条件	预期行为与系统输出	测试目的
玩家从高处坠落（y_vel > 0），且玩家包围盒下半部与怪物上半部发生重叠空间	触发 Enemy::stomped()，敌人切换为被踩扁贴图，玩家获得 jump_vel * 0.5 的向上回弹初速度	验证动态实体间的 AABB 相对位置计算与攻击判定有效性
大马里奥状态（isBig=true）下，纵向从下方撞击普通环境砖块	从容器中 removeAt 该砖块对象，执行 delete 回收内存，得分增加，并发播放物理碎砖音效	验证多态虚函数的事件分发及堆区内存的安全释放操作
玩家掉入关卡深渊断层（马里奥绝对坐标 y > C::SCREEN_H）	强制捕获坐标越界异常，冻结玩家输入源，切断主 BGM，调用 goDie() 触发死亡并重启整个场景	验证全局环境边界侦测机制及内存对象清空重载的稳定性
蘑菇道具从问号箱弹出，受重力落地并沿 X 轴向右平移，迎面撞击到场景静态水管	触发环境物理检测逻辑，强制执行动量系数翻转 x_vel *= -1；，蘑菇无缝衔接向左反向平移	验证非玩家控制实体（NPC）的 AI 自动寻路折返与边界阻挡系统

2. 缺陷修复与系统性能复盘

在中间测试版本的开发中，发现玩家在持续贴住垂直墙壁（如高耸的水管）下落时，偶尔会被卡在半空中发生悬浮假死。经过对运行时断点日志的追踪，发现原因是 Y 轴的落地检测宽度直接采用了原图像的物理宽度，当玩家紧贴墙壁时，碰撞盒边缘侵入了墙体几个像素，被算法误判为“双脚已经踩在墙壁正上方”。经过修复，在 Y 轴落地侦测时，引入了”adjusted(2, 0, -2, 0)”

方案，将侦测底盘的左右边缘强制向内收缩进行容错检测。成功消除了“蜘蛛侠式卡墙”的 Bug，所有测试案例最终均高标准通过。

六、 重大收获与技术反思

1. 数据驱动架构：JSON 解析的降维打击

在初期构思时，我曾试图在 C++ 源码中用数组或硬代码写死每一个水管、每一块砖的坐标。这导致只要稍微调整一下关卡设计，就需要花几分钟重新编译整个项目，耦合度极高且难以维护。后期我引入了 Qt 的 JSON 文档解析模块，实现了真正的“数据驱动开发（Data-Driven Architecture）”：

```
1 void GameWindow::setupBricksBoxesAndCoins() {
2     // 从 JSON 树中批量提取 brick 节点数组
3     QJsonArray bs = mapData["brick"].toArray();
4     for (auto v : bs) {
5         QJsonObject b = v.toObject();
6         // 动态提取配置文件的位姿信息，结合工程图元尺寸参数
7         int startX = b["x"].toInt();
8         int startY = b["y"].toInt();
9         int num = b.contains("brick_num") ? b["brick_num"].toInt() : 1;
10        // ... 根据计算出的矩阵绝对坐标，在堆区动态 new 出对象并挂载到场景
11    }
12 }
```

Listing 4: 动态解析 JSON 实例化对象构建内存拓扑

这种架构让我深刻领悟到现代软件工程的魅力：业务逻辑层（C++ 类）只负责“如果发生碰撞该怎么办”的物理法则，而持久数据层（JSON 文件）全权决定“地图的拓扑结构是什么样”。现在修改关卡只需修改本地 .json 文本文件即可实现秒级热更新，极大地提升了开发与测试的迭代效率。

2. 像素级实战：极其考验基本功的“抠坐标”纹理映射

本项目没有使用现成的散装单张图片素材，而是引入了业界 2D 游戏标准的“精灵图集（Sprite Sheet）”机制。所有的主角、怪物、物品动画帧全都紧凑地排列在一张超大的 .png 材质贴图中。这迫使我必须深入掌握图像的矩阵切片与纹理寻址技术：

```
1 // 裁剪并在内存中无损放大宝箱动画的 3 个关键循环帧
2 QPixmap f1 = sheet.copy(384, 0, 16, 16).scaled(sWidth, sHeight);
3 QPixmap f2 = sheet.copy(400, 0, 16, 16).scaled(sWidth, sHeight);
4 QPixmap f3 = sheet.copy(416, 0, 16, 16).scaled(sWidth, sHeight);
```

```
5 frames << f1 << f2 << f3 << f2; // 压入动态帧队列，并利用 f2 实现视觉往  
   复闪烁
```

Listing 5: 基于相对坐标系的精灵图矩阵精准切片算法

在完成这项工作时，需要在图形软件中建立二维直角坐标系，逐一计算各个独立图元的起始像素坐标 (x, y) 和宽高边界 (w, h)。这种与最底层像素进行“贴身肉搏”的经历，让我对计算机图形学中的纹理映射 (Texture Mapping) 原理和显存优化策略 (所有资源共用一张大图极大降低了 Draw Call 开销和 IO 换页频率) 有了透彻且深刻的理解。

3. 内存泄漏的梦魇与虚析构函数的救赎

在实现关卡重置 (“resetLevel”) 以重开游戏的功能时，由于使用了基类指针的多态容器 (如 “QList<TileItem*>”) 来统一管理和调度所有派生类地块，经历了一次惨痛的系统级内存溢出教训。

```
1 for (auto it : solidItems) {  
2     scene->removeItem(it); // 将图元从渲染树摘除  
3     delete it; // 多态释放的核心触发点  
4 }  
5 solidItems.clear(); // 清空悬空野指针
```

Listing 6: 多态架构下必须依靠虚函数保障的内存安全回收

在项目早期，由于未在基类中显式声明虚析构函数，导致执行 ‘delete it’ 动态卸载对象时，C++ 编译器静态绑定到了基类的空析构函数。派生类内部 ‘new’ 出的大量特效粒子、音频句柄和长动画帧列表内存完全处于 “被遗忘” 状态，造成严重的内存泄露。将基类添加 ‘virtual ~StaticItem()’ 后，内存使用量曲线立刻变得平滑稳定，这也是对 C++ 面向对象核心内存体系最深刻的一次实战洗礼。

七、 AI 工具使用披露声明

根据课程要求，本报告详细披露开发过程中的人工智能辅助使用记录：

- **AI 工具名称及具体版本：** Gemini 1.5 Pro
- **具体使用场景与用途：**
 1. **底层内存架构优化：**在排查上述基类析构引发的内存泄漏时，输入了 Qt QGraphicsScene 高频 removeItem 调用的崩溃日志，AI 辅助指出了多态体系下未声明虚析构的悬空指针风险，并给出了解决方案。
 2. **数学建模与物理推演验证：**提供了关于离散化牛顿运动学在固定帧率 (60FPS) 下的积分累加数学精度分析，协助推算出了能保证最佳跳跃手感的反重力常量 C::ANTI_GRAVITY。

3. **学术报告排版工程：**将零散撰写的日常开发手记和繁杂的代码片段，转化为符合规范的专业 LaTeX 自动化排版源代码。

- **具体使用位置在项目中的体现：**主要应用于游戏主引擎 ‘GameWindow.cpp’ 的垃圾回收（Garbage Collection）防内存泄漏策略重构；以及本份实验报告的整体 LaTeX 代码框架搭建、封面排版对齐与表格环境混排。